```
            }
        }
    }
};
```

That was quite a workout, but you got there in the end, and the stack is better for it. By using more relaxed operations in a carefully thought-through manner, the performance is improved without impacting the correctness. As you can see, the implementation of `pop()` is now 37 lines rather than the 8 lines of the equivalent `pop()` in the lock-based stack of listing 6.1 and the 7 lines of the basic lock-free stack without memory management in listing 7.2. As we move on to look at writing a lock-free queue, you'll see a similar pattern: lots of the complexity in lock-free code comes from managing memory.

7.2.6. Writing a thread-safe queue without locks

A queue offers a slightly different challenge to a stack, because the `push()` and `pop()` operations access different parts of the data structure in a queue, whereas they both access the same head node for a stack. Consequently, the synchronization needs are different. You need to ensure that changes made to one end are correctly visible to accesses at the other. However, the structure of `try_pop()` for the queue in listing 6.6 isn't actually that far off that of `pop()` for the simple lock-free stack in listing 7.2, so you can reasonably assume that the lock-free code won't be that dissimilar. Let's see how.

If you take listing 6.6 as a basis, you need two `node` pointers: one for the `head` of the list and one for the `tail`. You're going to be accessing these from multiple threads, so they'd better be atomic in order to allow you to do away with the corresponding mutexes. Let's start by making that small change and see where it gets you. The following listing shows the result.

Listing 7.13. A single-producer, single-consumer lock-free queue